

STFT Frontend Mismatch in Pretrained Audio Spectrogram Transformers: Analysis and Adaptation

Julia Wilkins¹, Wei-Ching Lin², Abinaya Kumar², Shabnam Ghaffarzadegan², Juan Pablo Bello¹

¹ New York University, USA

² Bosch Research, USA

Abstract—Pretrained audio spectrogram transformers are widely used as backbones of machine listening systems for sound event classification. However, they are typically trained using spectrograms with a fixed Short-Time Fourier Transform (STFT) configuration, implicitly binding the model to the time-frequency resolution seen during pretraining. In practice, adapting these systems to new domains or computational constraints often requires different STFT frontend parameters. In this work, we conduct a controlled study of the sensitivity of the foundational Audio Spectrogram Transformer to variations in STFT window and hop size, showing that audio classification performance degrades substantially as frontend parameters deviate from pretraining settings. As a mitigation strategy, we introduce a lightweight conditioning mechanism using FiLM that enables a pretrained model to adapt to variable input configurations using limited additional training data. Our method generalizes well to unseen STFT configurations, improving model robustness without sacrificing performance in the original setting.

Index Terms—Audio Spectrogram Transformers, Short-Time Fourier Transform, FiLM, Pretrained Model Adaptation, Audio Classification

1. INTRODUCTION

Pretrained transformer-based models have become foundational to many machine listening systems, with the Audio Spectrogram Transformer (AST) [1] and its variants [2]–[5] emerging as widely adopted backbones in the audio domain due to strong performance on audio classification benchmarks [6], [7] and ability to capture long-range temporal structure [8]. AST operates on log-Mel spectrograms as input, computed via the Short-Time Fourier Transform (STFT) [9], [10]. STFT parameters, such as window and hop length, control the time-frequency resolution of the resulting spectrogram for a given sampling rate [10]. This comes with fundamental tradeoffs: longer windows improve frequency resolution but reduce temporal precision and vice-versa; shorter hops improve temporal precision but increase computational costs. AST, like most pretrained machine listening models, is trained with a fixed STFT configuration, implicitly tying the model to the time-frequency resolution seen during pretraining.

In practice, when pretrained AST models are applied to new downstream tasks and datasets, the STFT hyperparameters are typically inherited from the pretraining pipeline rather than treated as a deliberate design decision. Yet audio datasets vary considerably in their acoustic characteristics: some tasks demand fine temporal resolution to capture transient sounds or rapid transitions [11]–[13], such as phoneme boundaries [14], [15] or species call onsets [16]–[18], while computational constraints in low-latency or resource-limited settings may necessitate coarser representations [19], [20]. When these demands conflict with the pretrained frontend configuration, the result is what we refer to in this study as an *STFT frontend mismatch*: a systematic discrepancy between the time-frequency resolution assumed by the model and what the downstream dataset or task requires.

This mismatch is typically treated as an input shape incompatibility and addressed with superficial workarounds. Common approaches include cropping, padding, repeating, or interpolating the input spectrogram to match the pretrained model’s expected dimensions, which can distort the time-frequency structure or discard information

[4], [21]. Alternatively, one could fine-tune or fully retrain the model on a new STFT configuration, but beyond the substantial data and compute requirements (e.g., AST was trained on 2 million 10s clips for 600 hours on 4 GPUs [1], [4]), this simply shifts the constraint to a different fixed configuration and does not ensure robustness to other unseen resolutions.

Prior work has been done to improve the flexibility of AST to changes in spectrogram patch size [22] and audio input length or temporal resolution [23]. The former uses variable patch sizes to proportionally adjust patch embedding weights during training [22]. The latter introduces a sequence packing method with masked self-attention to allow AST to handle variable-length inputs and temporally compressed resolutions [23]. While promising, these approaches still require re-training or fine-tuning a significant portion of the original AST due to underlying architecture changes. Further, neither approach directly investigates AST’s inflexibility to changes in both window and hop length simultaneously. Other similarly motivated works propose a module for learning the optimal temporal resolution of a spectrogram for classification tasks [13], or designing CNN layers that generalize across sampling rates for source separation [24]; the prior focuses on resolution selection during model training rather than pretrained model adaptation and the latter on sampling rate invariance alone. We address these limitations via the following contributions in this work:

- We present a systematic analysis of the sensitivity of pretrained AST to STFT frontend mismatch within a controlled study, showing substantial audio classification performance degradation as window and hop length deviate from pretraining settings.
- We propose a lightweight FiLM-based conditioning mechanism that enables pretrained AST to adapt to variable STFT window and hop lengths with minimal additional training.
- We show that our adaptation approach improves audio classification performance across unseen frontend settings while maintaining strong performance in the original configuration.
- We conduct a data efficiency study and show that substantial performance improvements can be achieved with as few as 1,000 additional training samples with our method.

For the scope of this work, we analyze the original AST model [1] as a widely adopted backbone for audio classification tasks. We use AudioSet [7] as an in-domain dataset to limit additional domain shift effects. We use STFT window and hop length as the axes of frontend variation, controlling for the sampling rate and other STFT hyperparameters to reduce confounding factors.

2. CHARACTERIZING STFT FRONTEND SENSITIVITY

To quantify AST’s sensitivity to STFT frontend mismatch, we first perform zero-shot evaluation using a pretrained Audio Spectrogram Transformer [1] (AST) across a range of STFT window and hop lengths. For each configuration, we compute audio classification performance on the AudioSet [7] evaluation set, and measure this relative to performance under the settings used during pretraining. The

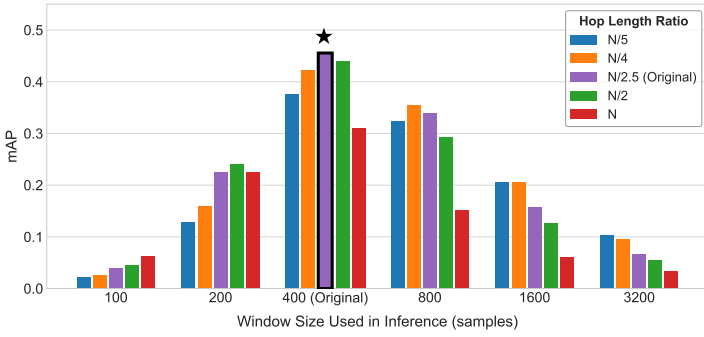


Fig. 1: Audio classification performance using AST degrades significantly as window and hop length parameters deviate from the original configurations used in pretraining (denoted with $\star$).

findings in Figure 1 illustrate our core motivating observation: AST performance degrades significantly when the window and hop length of the STFT used to create the input spectrogram differ from those used during pretraining. The original pretraining STFT configuration ($\star$) achieves the highest mean average precision (mAP) at 0.456, and classification performance consistently declines as window and hop length diverge from this matched setting. Reducing the window size by only 12.5 ms while maintaining the original window-hop length ratio yields over a 50% relative decrease in mAP ($0.456 \rightarrow 0.224$). Under more extreme shifts in time-frequency resolution such as with a window size of 100 samples, or a hop size equal to the window size (N), performance often falls below 0.1.

This analysis provides a systematic characterization of AST’s sensitivity to frontend mismatch, an effect that has not been explicitly quantified previously in terms of STFT window and hop length variation in pretrained AST models. This also establishes the zero-shot degradation baselines against which we will measure our adaptation method, and directly motivates the need for an adaptation strategy that allows pretrained AST models to operate more robustly across varying frontend configurations without large-scale re-training.

3. STFT FRONTEND ADAPTATION

To mitigate the degradation identified in Section 2, we propose a lightweight framework for variable frontend adaptation in pretrained AST models using FiLM conditioning [25]. FiLM is well suited to this problem because it enables conditional modulation of intermediate model representations using only a small number of additional learnable parameters, and has proven successful in contexts such as multi-modal source separation [26], [27], speech recognition [28], and speech robustness to temporal changes [29]. We build on the original AST [1] as a simple, interpretable backbone for studying STFT frontend adaptation in isolation, without additional architectural or data augmentation changes.

3.1. Audio Spectrogram Transformer

AST [1] takes a log-Mel spectrogram $x \in \mathbb{R}^{f \times t}$ of an audio clip as input, where f is the number of Mel-frequency bins and t is the number of temporal frames determined by the STFT window and hop size. The spectrogram is split into a sequence of overlapping patches, which are then flattened to a series of 1D patch embeddings via a linear projection. We denote the patch embeddings as $E \in \mathbb{R}^{T \times 768}$, where T is the number of patches and 768 is the embedding dimension. A $[\text{CLS}]$ token is appended to the beginning of the patch embedding sequence, and a learnable positional encoding $P \in \mathbb{R}^{T \times 768}$ is added to each patch embedding. This sequence is the input to a 12-layer

Transformer encoder [8]. The $[\text{CLS}]$ output of the transformer is fed through a linear classifier for the supervised audio classification task. AST is originally trained on AudioSet-unbalanced [7] and uses pretrained weights adapted from ViT [30].

3.2. Multi-scale Spectrogram Configuration Training

To isolate the effect of changes to the time-frequency resolution of the input log-Mel spectrogram, we vary only the STFT window length (N) and hop length (h) in our study. We keep all other spectrogram parameters fixed, namely sample rate (16 kHz), number of Mel-frequency bins (128), and window type (Hann), and leave variation in these parameters for future studies.

During large-scale pretraining, AST uses a window length of $N=400$ and hop length of $h=160$, in samples at 16 kHz. This corresponds to 25ms and 10ms, respectively, with a window-hop ratio of $N/2.5$. We probe window lengths logarithmically spaced around (but excluding) the reference of 400 samples, and a set of hop lengths proportional to window length. Specifically, we define the set of possible window lengths $\mathcal{N} \in \{100, 200, 800, 1600, 3200\}$ samples ($\{6.25, 12.5, 50, 100, 200\}$ ms, equivalently). For each window length $N \in \mathcal{N}$, we consider a set of hop lengths as a function of N : $\mathcal{H}(N) \in \{N/5, N/4, N/2, N\}$, varying the window-hop ratio while preserving relative scale across window lengths. We do not include the original window and hop length parameters ($N = 400, h = N/2.5$) in the set of training configurations, as we aim to train and evaluate using fully unseen parameters.

We create a 5-fold cross-validation protocol over the set of window lengths $\mathcal{N}$. In each fold, three window lengths are used as options for training, one for validation, and one for testing. All subsets are disjoint. For example, one fold uses window lengths $\{100, 1600, 3200\}$ for training, $\{800\}$ for validation, and $\{200\}$ for test. Across the five folds, each window length in $\mathcal{N}$ is used exactly once as the held-out test configuration, ensuring that performance is evaluated on all window lengths under unseen conditions. During training, for each mini-batch, we randomly sample a window length N from the training subset of $\mathcal{N}$, and then randomly sample hop length $h \in \mathcal{H}(N)$. The selected configuration $\mathcal{C} = \{N, h\}$ is used to parameterize the STFT for that batch, thus exposing the model to multiple time-frequency resolutions throughout training. This paradigm is loosely akin to the Multi-Scale Spectral Loss [31], [32], but serves a different purpose, as a mechanism used to generate variable input representations and corresponding conditioning signals vs. a reconstruction-driven objective.

3.3. Conditioning with FiLM

To adapt pretrained, frozen AST to unseen spectrogram configurations using minimal additional training parameters and limited training data, we propose to modulate the pretrained AST model using Feature-wise Linear Modulation (FiLM) [25]. FiLM applies feature-wise affine transformations to intermediate layers of a network given conditioning input. With FiLM, conditioning inputs are mapped to scaling and shifting coefficients via a learnable sub-network. These coefficients can then be directly applied as transformations to modulate the features of another network, taking the standard form: $\text{FiLM}(\mathbf{x}) = \gamma(\mathbf{z}) \odot \mathbf{x} + \beta(\mathbf{z})$, where $\mathbf{x}$ is the input to the FiLM layer and γ and β are scaling and shifting vectors that depend on the conditioning input $\mathbf{z}$.

For our application, the conditioning information is the window and hop length, chosen as the configuration for a particular training batch and parameterized as $\mathbf{z} = [\log(N), \log(h)]$. We observed that using the logarithm of the window and hop size vs. the raw values slightly improved training stability and performance, likely because of the multiplicative nature of changes in time-frequency resolution.

We freeze the entire AST model (≈ 87 million parameters) and train only the additional FiLM layers (198, 528 parameters) for our experiments. The FiLM sub-network contains two linear layers with a hidden dimension of 128 and GeLU activation [33], projecting $\mathbf{z}$ to dimension 1536 such that it can be split in half to form scaling and shifting parameters γ and β . We experiment with applying FiLM along the embedding dimension at three different points in the pretrained AST model ($\mathbf{x}$ in the FiLM equation above): (1) to the patch embeddings (FiLM-PatchEmb), (2) the positional encoding (FiLM-PosEnc), and (3) within each layer of the Transformer encoder (FiLM-Encoder). Notably, in each of these settings the FiLM network contains the same number of parameters due to its feature-wise nature.

When we use FiLM inside the Transformer encoder layers, we learn a single set of parameters and share that across each layer of the encoder. Additionally, in this setting we experiment with different FiLM locations *within* the Transformer layers: after initial layer normalization, after multi-head attention, after layer normalization prior to the feed-forward layer, and after the final feed-forward layer. We find applying FiLM prior to the feed-forward layer to be most effective and report results for this setting as our best FiLM-Encoder in Section 5.

4. EXPERIMENTAL DESIGN

4.1. Dataset

We choose AudioSet [7] for this study because it was the pretraining dataset for AST [1], allowing us to isolate the effect of variable STFT frontend configurations without confounding domain shift effects. This also makes zero-shot evaluation reasonable. We use the balanced subset of AudioSet for all model training, to illustrate model adaptation with minimal additional training resources. This subset contains 19,644 audio clips multi-labeled across 527 sound event classes. For evaluation, we use the AudioSet evaluation set containing 17,979 clips¹. Each audio clip is 10 seconds long and sampled at 16 kHz, following the original AST. We randomly split the AudioSet balanced (train) dataset into training and validation subsets, containing 16,697 and 2,947 clips respectively (85:15 split). We use the full AudioSet evaluation set for testing throughout.

4.2. Model Adaptation Considerations

Because AST is trained on 10-second clips with a fixed STFT configuration, the learned positional encoding is tied to the resulting token length of the original patched spectrogram. Modifying the window or hop length changes the number of temporal frames and thus the number of patch embeddings, causing a shape mismatch when adding the pretrained positional encoding to the patch embeddings. In the original AST preprocessing, such mismatches are handled by zero-padding or cropping the input spectrogram. However, in our case this could lead to information loss especially when smaller hop sizes produce denser temporal representations (e.g., more frames). Instead, we interpolate or crop the pretrained positional encoding to match the incoming patch embedding sequence, following the adaptation strategy used in [1] to transfer ViT [30] positional encodings to AST. This is the only modified component of the original AST architecture in our proposed method, and is an inference-time scaling operation and not a trainable component.

¹The original balanced and evaluation AudioSet contain 22k and 20k clips, respectively. Due to changes in privacy restrictions and removal of some videos, at the time of this study we were not able to obtain the entirety of the dataset.

4.3. Training Details

We follow the 5-fold multi-scale spectrogram cross-validation setup described in Section 3.2. The pretrained base model used throughout is the base AST (patch size 16). Each model is trained for 30 epochs with a batch size of 8, which takes 3–4 hours on 1×V100 GPU. For the few-shot adaptation experiments, we train for a maximum of 100 epochs using a random subset (of the full 20k) of 10, 100, 1k, 2k, 5k, and 10k samples. For all experiments, we use the Adam optimizer [34] with an initial learning rate of $2.5e-4$ and exponential decay. We do not use additional data augmentations as in the original AST in order to best isolate our adaptation effect. The training is supervised using a binary cross-entropy objective. Model selection is performed using lowest validation loss during training.

4.4. Evaluation Metrics

Each of the 5 folds as defined in 3.2 has exactly one held-out window size. For each model architecture and fold, we evaluate using the held-out window size and 4 window-hop ratios associated with this window size for inference. This yields 4 results per fold, totaling to $|C| = 4 \times 5 = 20$ window-hop configurations to evaluate per model architecture. We evaluate each of the 20 configurations above in terms of mean average precision (mAP) per the original multi-label evaluation protocol for classification with AudioSet [7] used in AST.

4.5. Baseline Methods

We use the pretrained AST model, frozen without additional training, as the **zero-shot** baseline when evaluated on STFT window and hop configurations *outside* the pretraining setting. This quantifies performance degradation introduced by frontend mismatch and serves as the primary reference point for measuring the benefits of our adaptation method across new configurations (Figure 1). The zero-shot baseline, evaluated using the *original* STFT window and hop length ($C = \{400, 160\}$) used in pretraining, serves as the **upper-bound**. This functions as a topline, reflecting performance without frontend mismatch.

As a lightweight adaptation alternative to FiLM, we provide **fine-tuning** (FT) baselines, in which either the classifier head (FT-ClfHead) or the positional encoding (FT-PosEnc) is fine-tuned and the rest of the pretrained AST is kept frozen. The fine-tuning is done using the identical multi-scale spectrogram training regime as is used for our proposed FiLM adapter. We do not include a fully fine-tuned or trained from scratch version of AST in the scope of this study, as we aim to provide computationally efficient adaptation strategies; we also suspect that training in this setting given the size of AST relative to the small size of AudioSet-balanced would not be particularly effective.

5. RESULTS

5.1. Adapting AST to unseen configurations with FiLM

Figure 2 (Left) shows audio classification performance (mAP) for our proposed FiLM variants compared to baseline methods, as a function of inference-time window length across window-hop ratios. The zero-shot AST baseline (red) shows a significant decline in performance from the matched upper-bound as window length deviates from the pretrained configuration ($N=400$), particularly at more extreme temporal resolutions (e.g., $N=100, 3200$). Fine-tuning baselines (blue) occasionally offer a slight improvement over zero-shot, especially when using larger window lengths. However, when the inference window length is closest to the original, fine-tuning slightly hurts performance relative to zero-shot. This suggests a tradeoff; fine-tuning using multiple frontend configurations may shift the learned

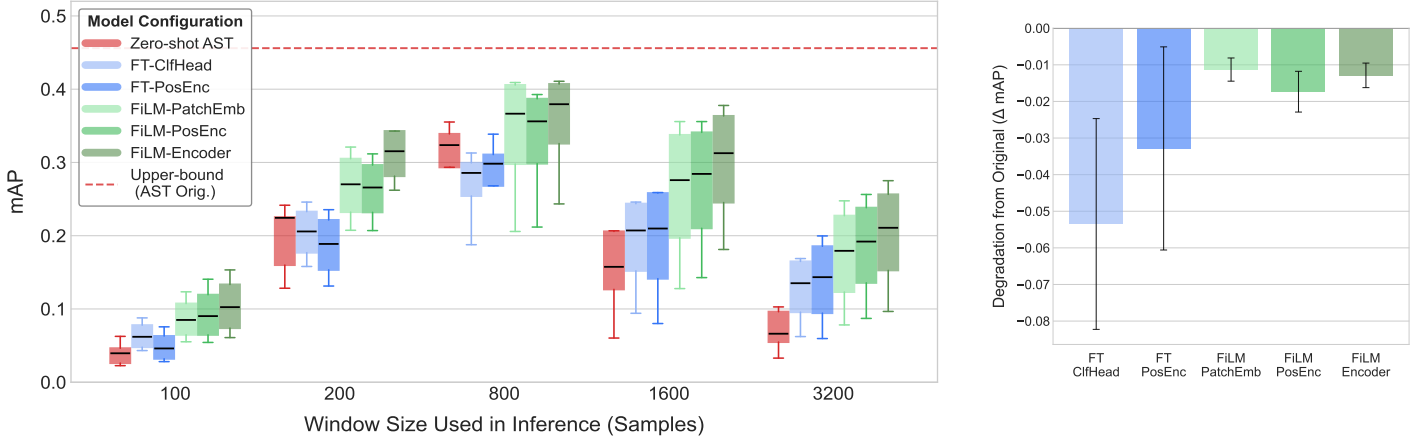


Fig. 2: (Left): Performance on unseen spectrogram configurations, in terms of mAP for each window length across window-hop ratios. (Right): Performance degradation relative to the original AST upper-bound (mAP = 0.456), shown across folds per model configuration.

distribution away from the pretrained optimum, reducing performance in matched or near-matched settings. We do not observe a significant difference between the effects of fine-tuning the classification head vs. the positional encoding.

In contrast, our FiLM-conditioned models (green) demonstrate consistent improvement across all evaluated unseen window lengths. The FiLM variants outperform fine-tuning baselines throughout and substantially improve upon the degradation observed in the zero-shot setting. We find that applying FiLM within the Transformer encoder layers (specifically prior to each feed-forward layer) is the most effective strategy. At $N=1600$, this configuration (FiLM-Encoder) nearly doubles zero-shot performance ($0.166 \rightarrow 0.313$, +14.7pp) in terms of median mAP across hop lengths per window. For window lengths closer to the pretrained setting ($N=200, 800$), FiLM still yields substantial gains, improving over zero-shot by +12.3 and +7.1pp, respectively. Notably, these improvements occur in both interpolated and extrapolated settings (e.g., within-range and out-of-range of window lengths seen during adaptation training), demonstrating that FiLM conditioning offers a strong, generalizable approach to enhancing robustness to frontend mismatch in pretrained ASTs.

In general, we also observe a large spread in results across hop ratios per window length in Figure 2 (Left). The smallest ($N/5$) and largest (N) hop settings consistently exhibit the lowest performance, likely due to the scale of distribution shift or coarseness of time-frequency resolution for $h=N$ specifically.

5.2. Preserving performance in the original setting

Unlike conventional fine-tuning approaches, which typically adapt a pretrained model to a new data domain without prioritizing original-task performance, our objective is to enable a single pretrained model to operate robustly across *any* STFT window and hop length configuration $\mathcal{C}$. Thus, maintaining strong performance in the original setting is an important requirement for our adaptation strategy. To measure this, we evaluate performance of each model using the *original* frontend, and report this result relative to the upper-bound: $\Delta \text{mAP} = \text{mAP}_{\text{trained}} - \text{mAP}_{\text{upper}}$. We show in Figure 2 (Right) that all of the FiLM-conditioned model variants are able to preserve performance in the original setting, with our best FiLM-Encoder version yielding an average mAP of 0.443, only -0.013 from the upper-bound. The fine-tuning baselines suffer more when evaluated using original parameters, illustrating benefits of training a drop-in conditioning mechanism such as FiLM vs. traditional fine-tuning for frontend adaptation.

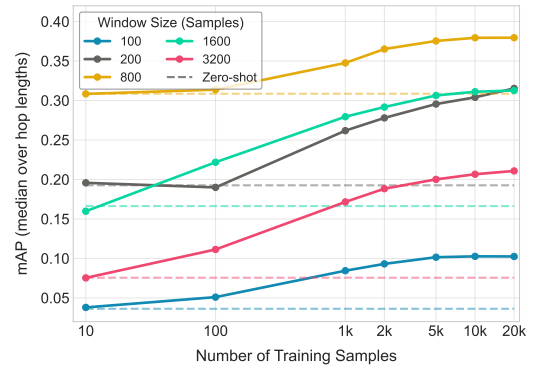


Fig. 3: Few-shot frontend adaptation as a function of number of training samples (FiLM-Encoder variant). Full dataset training is denoted at 20k samples.

5.3. How much data do you need for FiLM adaptation?

We evaluate the data efficiency of our best FiLM adaptation method (FiLM-Encoder) by varying the number of training samples used to learn the conditioning parameters. Figure 3 shows that substantial improvements over the zero-shot AST baseline are achieved with as few as 1,000 samples, with performance approaching that of the full dataset training (20k) using only 5k training samples across window lengths. While the trend is fairly consistent across window lengths, for larger windows, e.g., $N=1600$, a relative improvement of 33.7% (+5.6pp) over the zero-shot baseline is achievable with only 100 training samples.

6. CONCLUSION

This study is an exploration of STFT frontend mismatch within a controlled setting, limited to the original AST model and pretraining dataset with variation in STFT window and hop length. Future work will expand to other audio transformer models, diverse datasets, and other parameters, e.g., number of Mel filters and sampling rate, as well as tasks beyond classification where sensitivity to time-frequency resolution is even more critical. Within this setting, we present a systematic analysis of STFT frontend mismatch in a pretrained Audio Spectrogram Transformer, and propose a lightweight FiLM-based conditioning strategy to mitigate its effects. Our findings show that audio classification performance degrades substantially as window and hop length deviate from pretraining settings, and that our approach recovers much of this degradation while preserving performance in the original configuration using as few as 1,000 training samples.

REFERENCES

- [1] Y. Gong, Y.-A. Chung, and J. Glass, “Ast: Audio spectrogram transformer,” *arXiv preprint arXiv:2104.01778*, 2021.
- [2] Y. Gong, C.-I. Lai, Y.-A. Chung, and J. Glass, “Ssast: Self-supervised audio spectrogram transformer,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 10, 2022, pp. 10 699–10 709.
- [3] K. Koutini, J. Schlüter, H. Eghbal-zadeh, and G. Widmer, “Efficient training of audio transformers with patchout,” in *Interspeech 2022*, 2022, pp. 2753–2757.
- [4] K. Chen, X. Du, B. Zhu, Z. Ma, T. Berg-Kirkpatrick, and S. Dubnov, “Hts-at: A hierarchical token-semantic audio transformer for sound classification and detection,” in *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2022, pp. 646–650.
- [5] A. Baade, P. Peng, and D. Harwath, “Mae-ast: Masked autoencoding audio spectrogram transformer,” *arXiv preprint arXiv:2203.16691*, 2022.
- [6] K. J. Piczak, “Esc: Dataset for environmental sound classification,” in *Proceedings of the 23rd ACM international conference on Multimedia*, 2015, pp. 1015–1018.
- [7] J. F. Gemmeke, D. P. Ellis, D. Freedman, A. Jansen, W. Lawrence, R. C. Moore, M. Plakal, and M. Ritter, “Audio set: An ontology and human-labeled dataset for audio events,” in *2017 IEEE international conference on acoustics, speech and signal processing (ICASSP)*. IEEE, 2017, pp. 776–780.
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, E. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [9] J. Dennis, H. D. Tran, and H. Li, “Spectrogram image feature for sound event classification in mismatched conditions,” *IEEE signal processing letters*, vol. 18, no. 2, pp. 130–133, 2010.
- [10] M. Müller, *Fundamentals of music processing: Audio, analysis, algorithms, applications*. Springer, 2015, vol. 5.
- [11] P. Khunarsal, C. Lursinsap, and T. Raicharoen, “Very short time environmental sound classification based on spectrogram pattern matching,” *Information Sciences*, vol. 243, pp. 57–74, 2013.
- [12] M. Huzaifah, “Comparison of time-frequency representations for environmental sound classification using convolutional neural networks,” *arXiv preprint arXiv:1706.07156*, 2017.
- [13] H. Liu, X. Liu, Q. Kong, W. Wang, and M. D. Plumbley, “Learning temporal resolution in spectrogram for audio classification,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 12, pp. 13 873–13 881, Mar. 2024. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/29294>
- [14] D. T. Toledano, M. P. Fernández-Gallego, and A. Lozano-Diez, “Multi-resolution speech analysis for automatic speech recognition using deep neural networks: Experiments on timit,” *PloS one*, vol. 13, no. 10, p. e0205355, 2018.
- [15] Y. Zhu and T. Falk, “Wavrx: a disease-agnostic, generalizable, and privacy-preserving speech health diagnostic model,” *IEEE Journal of Biomedical and Health Informatics*, 2024.
- [16] J. S. Ulloa, A. Gasc, P. Gaucher, T. Aubin, M. Réjou-Méchain, and J. Sueur, “Screening large audio datasets to determine the time and space distribution of screaming piha birds in a tropical forest,” *Ecological informatics*, vol. 31, pp. 91–99, 2016.
- [17] V. Lostanlen, K. Palmer, E. Knight, C. Clark, H. Klinck, A. Farnsworth, T. Wong, J. Cramer, and J. P. Bello, “Long-distance detection of bioacoustic events with per-channel energy normalization,” *arXiv preprint arXiv:1911.00417*, 2019.
- [18] E. C. Knight, S. Poo Hernandez, E. M. Bayne, V. Bulitko, and B. V. Tucker, “Pre-processing spectrogram parameters improve the accuracy of bioacoustic classification using convolutional neural networks,” *Bioacoustics*, vol. 29, no. 3, pp. 337–355, 2020.
- [19] I. Martín-Morató, F. Paissan, A. Ancilotto, T. Heittola, A. Mesaros, E. Farella, A. Brutti, and T. Virtanen, “Low-complexity acoustic scene classification in dcase 2022 challenge,” 2022. [Online]. Available: <https://arxiv.org/abs/2206.03835>
- [20] J. W. Yeow, E.-L. Tan, J. Bai, S. Peksi, and W.-S. Gan, “Real-time sound event localization and detection: Deployment challenges on edge devices,” *arXiv preprint arXiv:2409.11700*, 2024.
- [21] Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, and M. D. Plumbley, “Panns: Large-scale pretrained audio neural networks for audio pattern recognition,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 28, pp. 2880–2894, 2020.
- [22] J. Feng, M. H. Erol, J. S. Chung, and A. Senocak, “Flexiast: Flexibility is what ast needs,” *arXiv preprint arXiv:2307.09286*, 2023.
- [23] —, “Elasticast: an audio spectrogram transformer for all length and resolutions,” *arXiv preprint arXiv:2407.08691*, 2024.
- [24] K. Saito, T. Nakamura, K. Yatabe, and H. Saruwatari, “Sampling-frequency-independent convolutional layer and its application to audio source separation,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 30, pp. 2928–2943, 2022.
- [25] E. Perez, F. Strub, H. De Vries, V. Dumoulin, and A. Courville, “Film: Visual reasoning with a general conditioning layer,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, no. 1, 2018.
- [26] X. Liu, H. Liu, Q. Kong, X. Mei, J. Zhao, Q. Huang, M. D. Plumbley, and W. Wang, “Separate what you describe: Language-queried audio source separation,” *arXiv preprint arXiv:2203.15147*, 2022.
- [27] X. Liu, Q. Kong, Y. Zhao, H. Liu, Y. Yuan, Y. Liu, R. Xia, Y. Wang, M. D. Plumbley, and W. Wang, “Separate anything you describe,” *IEEE Transactions on Audio, Speech and Language Processing*, vol. 33, pp. 458–471, 2024.
- [28] D.-H. Yang and J.-H. Chang, “Film conditioning with enhanced feature to the transformer-based end-to-end noisy speech recognition,” in *Interspeech*, 2022, pp. 4098–4102.
- [29] D. A. Wisnu, R. E. Zezario, S. Rini, F.-R. Li, Y.-T. Peng, H.-M. Wang, and Y. Tsao, “Stsm-film: A film-conditioned neural architecture for time-scale modification of speech,” *arXiv preprint arXiv:2510.02672*, 2025.
- [30] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, “An image is worth 16x16 words: Transformers for image recognition at scale,” *arXiv preprint arXiv:2010.11929*, 2020.
- [31] J. Engel, L. Hantrakul, C. Gu, and A. Roberts, “Ddsp: Differentiable digital signal processing,” *arXiv preprint arXiv:2001.04643*, 2020.
- [32] S. Schwär and M. Müller, “Multi-scale spectral loss revisited,” *IEEE Signal Processing Letters*, vol. 30, pp. 1712–1716, 2023.
- [33] D. Hendrycks and K. Gimpel, “Gaussian error linear units (gelus),” *arXiv preprint arXiv:1606.08415*, 2016.
- [34] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2014.